# BCSE408L – CLOUD COMPUTING

## FINAL PROJECT REPORT

Academic Year 2026–2027 | VIT Chennai

## Submission Information

| | |
|---|---|
| Project Title | ServiceDesk AI — A Cloud-Native WhatsApp Integration Platform with an AI-Assisted Support Desk |
| Team Member 1 – Name & Reg. No. | Aarush Jagannathan — 23BLC1211 |
| Team Member 2 – Name & Reg. No. | Gomatheswar M — 23BLC1195 |
| Faculty / Slot | Dr. M Dinakaran / D1+TD1 |
| Public Live URL | https://servicedesk-ai-two.vercel.app |
| GitHub Repository URL | https://github.com/ItzGOLLY/servicedesk-ai |
| Submission Deadline | 28/10/2026 |

*Important: This report is a supporting document for project evaluation. Project marks are based on the reviews and final demonstration completed during the semester, and not on this report submission alone.*

## 1. Abstract

A small business that receives customer complaints across WhatsApp, email and telephone has no reliable record of what was reported, who is handling it, or whether it was ever resolved. Commercial helpdesk products solve this but are priced per agent per month and assume a dedicated support team, placing them beyond a three-person business.

ServiceDesk AI addresses this in two layers. The upper layer is a cloud-native Software-as-a-Service support desk: every complaint becomes a ticket with a reference, an owner, a category, a priority and a tracked status, visible to the customer, actionable by a support agent and measurable by an administrator. The lower layer — and the substantial addition since the mid-semester reviews — is a WhatsApp integration platform: other businesses connect their own WhatsApp Business numbers through Meta's Embedded Signup, and the system routes, stores and serves each business's messages in isolation. This is the model operated commercially by Twilio and AiSensy, and the support desk is simply the first application running on it.

Three user roles are implemented — Customer, Support Agent and Administrator — with role-based access control enforced in the API rather than the interface. Major features include a guarded six-state ticket lifecycle, threaded conversations with staff-only internal notes, full-text search with filtering and pagination, persistent notifications, three role-specific dashboards, seven exportable reports, an administration module with an audit log, a two-way WhatsApp channel, an AI assistance layer that classifies and drafts but never messages a customer without agent approval, and a knowledge base answering questions from indexed articles using vector similarity search.

The application is deployed on public cloud infrastructure: the React frontend on Vercel's global CDN, the Express/TypeScript API on Render from a version-controlled Blueprint, and a managed PostgreSQL instance on Neon. Correctness is evidenced by 193 automated tests executed against a real PostgreSQL database using the production migration files, all passing, and run by continuous integration on every push.

**Keywords:** Cloud Computing, Software as a Service, Platform as a Service, Multi-Tenancy, Customer Support, Ticket Management, Role-Based Access Control, REST API, PostgreSQL, pgvector, React, Node.js, WhatsApp Business Platform, Embedded Signup, Retrieval-Augmented Generation, Graceful Degradation.

# 2. Introduction

## 2.1 Background

Cloud computing has changed how small organisations obtain software. Capabilities that once required a server room, a database administrator and a capital purchase are now consumed over the internet, billed by usage and maintained by the provider. The service models are conventionally described in three layers: Infrastructure as a Service, where virtual machines and storage are rented; Platform as a Service, where an application is deployed onto a managed runtime; and Software as a Service, where finished software is used directly through a browser.

This project occupies all three at once, which is what makes it a suitable subject for a cloud computing course. The delivered product is Software as a Service: a business signs in through a browser and uses a support desk without installing anything. It is built on Platform as a Service: the frontend and the API are deployed to managed platforms that own the runtime, scaling and transport security. It stores its data in a managed Database as a Service instance, so no member of the team administers a database server.

## 2.2 Purpose

The purpose is twofold. First, to give a small business the structure of a helpdesk — ticket identity, ownership, visible status, access control and measurement — without the pricing or configuration burden of an enterprise product. Second, to meet customers on the channel they already use. In India a small business's customers do not open a support portal; they send a WhatsApp message. The system was therefore built so that WhatsApp is a first-class intake channel, and then generalised so that any business can connect its own WhatsApp number rather than only the one this deployment owns.

# 3. Problem Statement

The official specification states the problem in one sentence: *"Manage customer complaints and automate ticket handling."* Requirement analysis decomposed this into four concrete failures observed in small-business support.

| Observed failure | What actually happens | Addressed by |
|---|---|---|
| No single intake point | Requests arrive by WhatsApp, email and telephone and are never consolidated into one record | One ticket store with a generated reference; WhatsApp messages become tickets automatically |
| No ownership | Without an assigned owner a complaint is answered twice by different people, or not at all | An assignment field, an agent queue, and notification on assignment |
| No status or history | The customer cannot see progress, so they follow up repeatedly, generating further work | A tracked six-state lifecycle, a threaded conversation, a per-ticket timeline, and a STATUS keyword on WhatsApp |
| No measurement | The business cannot state how many issues arrived, in which categories, or how long they took | Role-specific dashboards and reports computed from live data |

*Table 3.1 — Decomposition of the problem statement*

**On the word "automate".** It admits two readings. Procedural automation — creating the record, assigning a reference, timestamping events, advancing status when an agent replies, notifying, acknowledging a WhatsApp message — is implemented in full and depends on no artificial intelligence. Intelligent automation — interpreting the content of a complaint — is also implemented, as classification, summarisation, drafting and knowledge-base answers, but with a deliberate boundary: the system automates the preparation of a response, never the decision to send one.

# 4. Existing System

Three approaches are in common use for handling customer complaints in a small business. Each was examined for the capabilities it provides and the point at which it fails.

### 4.1 Shared inbox or a personal WhatsApp number

- There is no ticket identity: a customer telephoning about an earlier message has no reference to quote.
- There is no ownership, so two staff members can reply to the same message independently.
- The only state available is read or unread, which cannot express “waiting for the customer”.
- History is scattered across threads and personal phones, and no reporting of any kind is possible.

### 4.2 Spreadsheet tracker

- It is updated by hand, so it drifts out of date almost immediately and is trusted less over time.
- It offers the customer nothing: there is no view they can access.
- It has no access control, and no record of who changed what.
- It becomes unusable beyond a few dozen rows — precisely when the business most needs it.

### 4.3 Commercial helpdesk platforms

Established products such as Zendesk, Freshdesk and Jira Service Management solve the problem thoroughly and are the correct choice for an organisation of sufficient size. They are priced per agent per month, so cost rises with team size rather than value obtained; they assume a dedicated support function with defined processes; they require substantial configuration before delivering any value; and WhatsApp is typically a paid add-on.

### 4.4 Comparison

| Capability | Shared inbox | Spreadsheet | Commercial helpdesk | ServiceDesk AI |
|---|---|---|---|---|
| Ticket reference | No | Manual | Yes | Yes |
| Assigned owner | No | Manual | Yes | Yes |
| Customer-visible status | No | No | Yes | Yes — web and WhatsApp |
| Access control by role | No | No | Yes | Yes, enforced in the API |
| Search and filtering | Limited | Limited | Yes | Yes, full-text |
| Reporting | No | Manual | Yes | Yes, from live data |
| Audit trail | No | No | Yes | Yes |
| Automatic classification | No | No | Partly | Yes, with offline fallback |
| WhatsApp intake | Informal | No | Paid add-on | Built in |
| Connect your own number | n/a | n/a | Enterprise tier | Yes — Embedded Signup |
| Cost at three agents | Free | Free | Recurring per agent | Free-tier hosting |

*Table 4.1 — Existing approaches compared with the proposed system*

# 5. Proposed System

### 5.1 Overview

ServiceDesk AI is a multi-tenant, cloud-hosted SaaS application. Every complaint becomes a ticket carrying an owner, a category, a priority and a tracked status. The customer can see it on the web or by WhatsApp, the

agent can act on it, and the administrator can measure it. Beneath that, the platform allows any onboarded business to attach its own WhatsApp Business number, with its data kept strictly separate from every other business on the deployment.

### 5.2 Workflow

1. A customer sends a WhatsApp message, or raises a ticket on the web. The ticket is stored immediately with status NEW and a reference such as SD-1024; on WhatsApp the customer receives an acknowledgement naming that reference within about a second.
2. Classification runs in the background after the database commit: category, priority, sentiment and a one-line summary. A failure here cannot affect the ticket.
3. The ticket enters the agent queue ordered by priority, filterable and searchable across every customer.
4. An agent takes ownership, moving the ticket to OPEN, and may ask the assistant for a summary, suggested resolution steps, a knowledge-base answer with citations, or a draft reply. The agent edits and sends; sending records the first-response time, moves the ticket to IN_PROGRESS, and delivers the reply to the customer's WhatsApp as plain text.
5. The customer is notified and can respond in the thread or the chat. A reply on a ticket that was WAITING_FOR_CUSTOMER returns it to IN_PROGRESS.
6. When the issue is fixed the agent marks the ticket RESOLVED. The customer confirms by closing it, on the web or with the CLOSE keyword, or reopens it if the problem persists.
7. Throughout, every change writes a timeline event and every privileged action writes an audit record. These feed the dashboards and reports.

### 5.3 What makes the system cloud-native

| Property | Realisation in this system |
|---|---|
| Managed data tier | PostgreSQL is consumed as a service with its own backups, pooling and availability guarantees. No team member patches or administers a database server. |
| Stateless application tier | The API holds no session state — sessions are signed tokens and all other state lives in the database — so the platform can run several replicas without sticky sessions. |
| Edge-distributed presentation tier | The frontend is a static build cached by a content delivery network close to the user, and scales independently of the API. |
| Infrastructure as code | render.yaml provisions the database and the API together, generates the JWT secrets and wires the connection string, so the deployment is a repeatable definition rather than a sequence of clicks. |
| Continuous delivery | A push to the default branch runs the test suite and redeploys both tiers automatically. |
| Multi-tenancy | One deployment serves many businesses, each with isolated data and its own externally connected messaging number. |
| Public reachability | The application is addressable from any browser on the internet and from any permitted WhatsApp number. There is no dependence on localhost. |

*Table 5.1 — Cloud-native properties*

### 5.4 Advantages over the existing approaches

- Every complaint acquires an identity, an owner and a visible status, removing the three principal failure modes of a shared inbox.
- Access is controlled by role and enforced by the API, so a customer cannot reach another customer's data even by calling the interface directly.
- Measurement is a by-product of normal use rather than a separate manual exercise.

- Classification orders the queue automatically, so an urgent billing failure is not buried behind a formatting question.
- Customers use WhatsApp, which requires no training, no sign-up and no application install.
- Operating cost at the scale of a small business is the cost of the hosting free tier rather than a recurring per-agent charge.

# 6. Requirement Analysis

## 6.1 Users identified

| User | Responsibilities and interest |
|---|---|
| Customer | Raises tickets on the web or by WhatsApp, follows their progress and replies within the thread. Sees only their own tickets. |
| Support Agent | Works a prioritised queue spanning every customer; updates status, priority, category and assignment; adds staff-only internal notes; uses the assistance layer. Replies reach WhatsApp customers automatically. |
| Administrator | Holds every agent capability and additionally manages users, roles and categories, views global reporting, reviews the audit log, and configures the WhatsApp connection and developer access. |
| Evaluator | Requires demonstrable satisfaction of every mandatory requirement, a reachable live URL and working credentials. |

*Table 6.1 — Users*

## 6.2 Constraints that shaped the design

| Constraint | Design consequence |
|---|---|
| The application must be publicly accessible; localhost is not acceptable | Three-tier deployment to managed platforms; no component assumes a local address |
| Both team members must be able to explain and modify any line | Deliberate avoidance of unnecessary abstraction; one rule in one place, such as the single lifecycle table |
| All work must be original | The design system and every component were written for this project; no user-interface kit or template was used |
| Free hosting tiers suspend idle services | A health endpoint suitable for scheduled pinging, and an interface that shows a loading state rather than appearing broken |
| No secret may be committed or exposed in the browser bundle | All secrets supplied as environment variables; the frontend holds exactly one public value, the API base URL |
| WhatsApp renders no Markdown | Every outbound message passes through a plain-text formatter |
| One deployment serves several businesses | Every tenant-owned table carries a tenant identifier that is never accepted from client input |

*Table 6.2 — Constraints and their consequences*

## 6.3 Assumptions

- Users have a modern browser and internet access.
- Staff accounts are created by an administrator rather than by self-registration.
- English is the only interface language.
- Ticket volumes are consistent with a small business — thousands of records rather than millions.
- The assistance provider, where configured, is reachable over HTTPS, and the system tolerates it not being so.

# 7. Functional Requirements

The ten requirements below are those stated in the project specification, elaborated into the specific behaviour implemented. Test identifiers refer to section 15.

| ID | Requirement | Implemented behaviour | Evidence |
|---|---|---|---|
| FR-1 | User registration and login | Registration with name, unique case-insensitive email and a password of at least eight characters containing a letter and a digit; bcrypt hashing at cost 12; sign in, refresh and sign out; self-registration always creates a Customer | 13 tests |
| FR-2 | Profile management | View and edit own name and telephone number; change password by supplying the current one; a user cannot change their own role or email address | Browser-verified |
| FR-3 | Role-based access control | Three roles; every protected route verifies the role before the handler executes; ticket handlers additionally verify record ownership; another customer's ticket returns 404 rather than 403; a deactivated account is refused immediately | 26 tests |
| FR-4 | CRUD operations | Create, read, update and delete for tickets, messages, users, categories and knowledge-base articles; deletion cascades to messages and timeline events; status changes are validated against the lifecycle table | 33 tests |
| FR-5 | Search and filtering | Filter by status, priority, category, assigned agent, sentiment, channel and date range; full-text search over subject, description and reference using a GIN index; paginated and sortable; a customer's results are always scoped to their own tickets | 7 tests |
| FR-6 | Dashboard | Three role-specific dashboards; every figure aggregated from the database at request time, with no hard-coded value anywhere in the codebase | Browser-verified |
| FR-7 | Notifications | Persistent notifications on assignment, reply and status change; unread badge; mark one or all read; WhatsApp customers additionally receive status changes on their phone | Tested |
| FR-8 | Report generation | Seven reports — by status, priority and category, sentiment distribution, agent workload, created-versus-resolved trend and resolution time by category — plus RFC 4180 CSV export with formula-injection protection | Browser-verified |
| FR-9 | Cloud database integration | All persistent data in a managed cloud PostgreSQL instance; schema applied by versioned migrations that run automatically on every deployment; the frontend never connects to the database | Whole suite |
| FR-10 | Administrator module | List, search and create users; change roles; activate and deactivate accounts; an administrator cannot remove their own administrative role; manage categories; review an audit log of every privileged action | 4 tests |

*Table 7.1 — Functional requirements and their evidence*

## 7.1 Additional capabilities implemented beyond the specification

| Capability | Description |
|---|---|
| WhatsApp channel | Customers raise and follow tickets entirely from WhatsApp using the keywords HELP, STATUS, NEW and CLOSE; agent replies are delivered back to the same chat as plain text |
| Integration platform | Other businesses connect their own WhatsApp Business numbers through Meta's Embedded Signup; one webhook serves every business, routed by the number the message arrived on |
| Developer access | Each business can issue API keys and register webhook endpoints, receiving signed event deliveries with retry and backoff |

| Capability | Description |
|---|---|
| AI assistance | Classification of category, priority and sentiment; conversation summaries; suggested resolution steps; agent-reviewed draft replies — none of which reaches a customer without an agent sending it |
| Knowledge base | Articles chunked, embedded and retrieved by vector similarity in PostgreSQL, answering questions with citations and reporting insufficiency rather than inventing an answer |

*Table 7.2 — Capabilities beyond the stated requirements*

## 8. Non-Functional Requirements

| Requirement | Concrete mechanism |
|---|---|
| Security | bcrypt at cost 12; a short-lived JSON Web Token held only in memory with a refresh token in an httpOnly cookie scoped to the authentication path; authorisation enforced in two layers; every input validated with Zod; parameterised SQL exclusively; Helmet security headers; a CORS allow-list rather than a wildcard; rate limiting; webhook signatures verified in constant time; identical responses for an unknown email and a wrong password; third-party access tokens encrypted at rest with AES-256-GCM; secrets redacted from logs |
| Scalability | A stateless API that permits horizontal replication; a bounded connection pool so replicas do not exhaust the database; server-side pagination on every list; a CDN-served frontend; message classification and outbound delivery kept off the request path |
| Availability | Managed services with their own redundancy; a health endpoint that checks both process and database; graceful shutdown on SIGTERM; the assistance layer degrades to a rule-based classifier rather than failing; a WhatsApp delivery failure never rolls back the ticket change that triggered it; migrations run before the server accepts traffic |
| Performance | An index on every column used as a filter or join; a GIN index for full-text search; an HNSW index for vector similarity; pagination throughout; client-side caching; classification deferred until after the database commit so ticket creation returns immediately |
| Usability | One hand-built design system; responsive from 375 pixels; explicit loading, empty and error states on every data view; plain-language error messages; WhatsApp replies in plain text because WhatsApp renders no formatting |
| Reliability | Database CHECK and foreign-key constraints enforcing invariants independently of application code; a single lifecycle transition table; transactions for multi-step writes; idempotent webhooks via a unique provider message identifier; an append-only audit log; 193 automated tests against a real database |

*Table 8.1 — Non-functional requirements*

## 9. System Architecture

The system is a three-tier web application with two external integrations. The browser and the customer's WhatsApp are the clients; the API is the only component that touches the database; and every external dependency sits behind an adapter with an offline implementation, so the application runs with none of them configured.

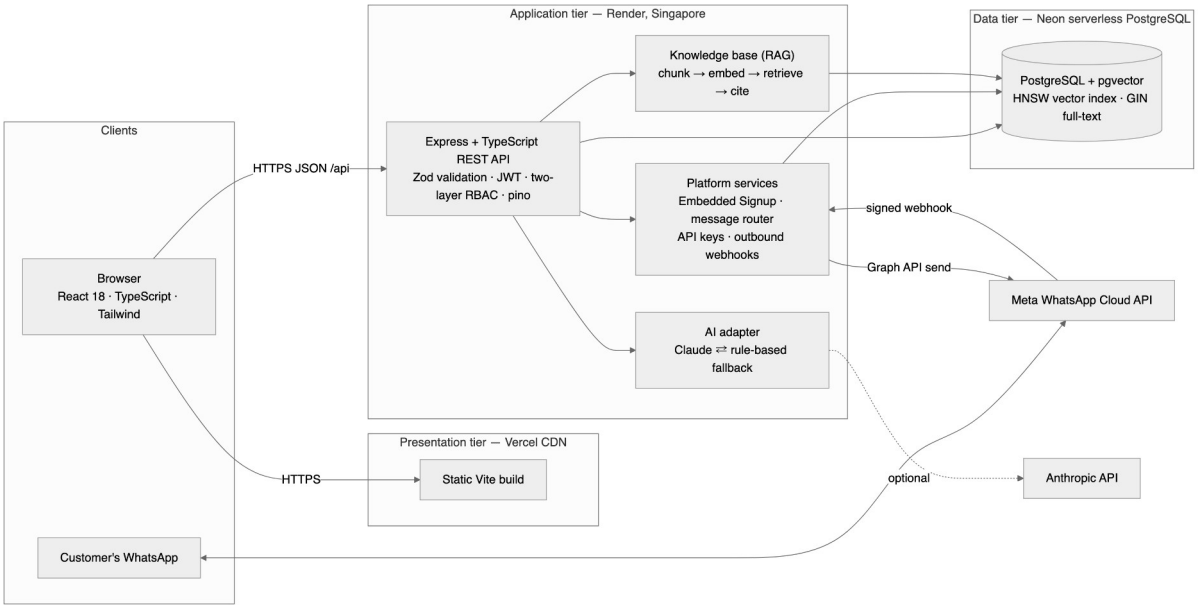



*Figure 9.1 — System architecture*

## 9.1 Components and responsibilities

| Tier | Technology | Responsibility |
|---|---|---|
| Presentation | React 18, TypeScript, Vite, Tailwind CSS, TanStack Query, React Router, Recharts | A role-aware single-page application. Holds the access token in memory only, never in browser storage; calls the API over HTTPS; never sees a secret or the database. |
| Application | Node.js 20, Express, TypeScript, Zod, jsonwebtoken, bcryptjs, pino, Helmet | The REST API. Validates every input, authenticates, authorises, enforces business rules, writes audit and timeline records, and orchestrates the AI and messaging adapters. |
| Data | PostgreSQL 16 with the pgvector extension; versioned SQL migrations | The single source of truth. Constraints enforce invariants independently of application code; a GIN index serves full-text search and an HNSW index serves vector similarity. |
| Integrations | Meta WhatsApp Cloud API; Anthropic Claude; an OpenAI-compatible embeddings endpoint | Each sits behind a small interface with an offline fallback, so no mandatory requirement depends on a third party being reachable. |

*Table 9.1 — Tiers and responsibilities*

## 9.2 The path of a request

1. The request passes the HTTP logger, which assigns a request identifier echoed in the response, then Helmet, the CORS allow-list, body parsing that retains a copy of the raw body for signature verification, and the rate limiter.
2. Authentication verifies the token and re-reads the user row, so a deactivated account is refused on its very next request rather than when the token happens to expire.
3. Authorisation checks the route's permitted roles; handlers that touch a specific record check ownership again inside the handler.
4. Validation parses the body, query and parameters with Zod; invalid input returns a 400 response carrying field-level errors.
5. The handler runs parameterised SQL through a shared pool; multi-step writes execute inside a transaction.
6. Side effects that must not block the response — classification, message delivery, notifications — run after the commit and record their own outcome.

7. Every response uses one envelope: a success flag with data and optional metadata, or a success flag with a structured error.

## 10. Cloud Architecture

The deployed topology is shown below exactly as it runs. Every box is a managed service; the team operates no server and patches no operating system.

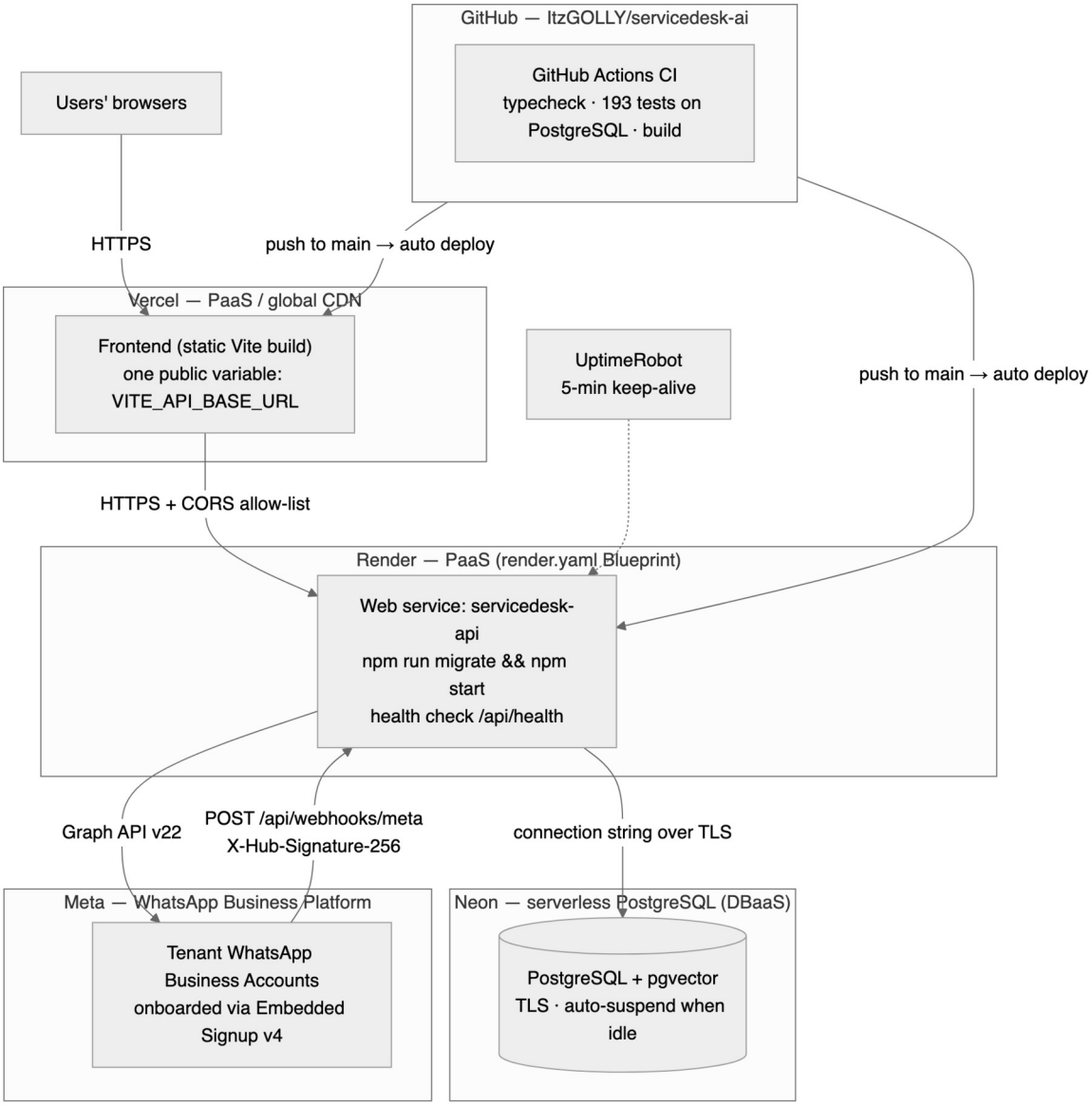



*Figure 10.1 — Cloud deployment architecture*

### 10.1 Cloud services used

| Layer | Service | Cloud model | Why this choice |
|---|---|---|---|
| Frontend hosting | Vercel | PaaS / CDN | A static build cached at the edge close to the user; automatic deployment on push; generous free tier |
| Application hosting | Render Web Service | PaaS | The platform owns the Node runtime, TLS termination and restarts, and supports infrastructure as code; its free web service requires no payment card |
| Database | Neon serverless PostgreSQL | DBaaS | A permanently free plan requiring no payment card, carrying the pgvector extension this project needs, with TLS and compute that suspends when idle |

| Layer | Service | Cloud model | Why this choice |
|---|---|---|---|
| Messaging | Meta WhatsApp Cloud API | SaaS API | The official WhatsApp Business Platform; supports multi-customer onboarding through Embedded Signup |
| Continuous integration | GitHub Actions | SaaS | Typecheck, the full test suite against a real PostgreSQL service container, and frontend and container builds on every push |
| Uptime monitoring | UptimeRobot | SaaS | A five-minute health-check ping, so the free instance does not sleep before an evaluation |

*Table 10.1 — Cloud services and resources*

### 10.2 Infrastructure as code

The file `render.yaml` at the repository root defines the database and the web service together: plan, region, the Node build, a start command that applies migrations before the server binds its port, the health-check path, automatically generated JWT secrets, the database connection string wired from the database resource, and every third-party secret marked so that it is entered once in the dashboard and never stored in the repository. During the first deployment the database defaulted to a different region from the API, so the internal connection string did not resolve; the fix was one line in this file and a re-apply, which is precisely the value of defining infrastructure in version control.

### 10.3 Security boundaries in the cloud

- The database is reached only over TLS, using a connection string held as a platform secret that is never committed to the repository.
- The browser holds exactly one configuration value, the API base URL. No AI key, messaging token or database credential ever reaches the client bundle.
- CORS is an exact-origin allow-list, which is required because the refresh cookie is sent with credentials.
- Inbound webhooks are signed by the provider and verified in constant time before any processing occurs.
- Third-party access tokens held on behalf of connected businesses are encrypted at rest; logs carry a short fingerprint, never the token.

## 11. Database Design

### 11.1 Why a relational database

A support desk is a set of entities with strict relationships and invariants: a message belongs to exactly one ticket; a ticket has exactly one customer; a status may only change along defined paths; a resolution timestamp only makes sense in a resolved state. PostgreSQL enforces all of this in the schema, so the invariants hold even if application code contains a defect. Full-text search and, through the pgvector extension, vector similarity search are available in the same engine, which avoided introducing a separate search or vector database.

### 11.2 Entity relationship diagram

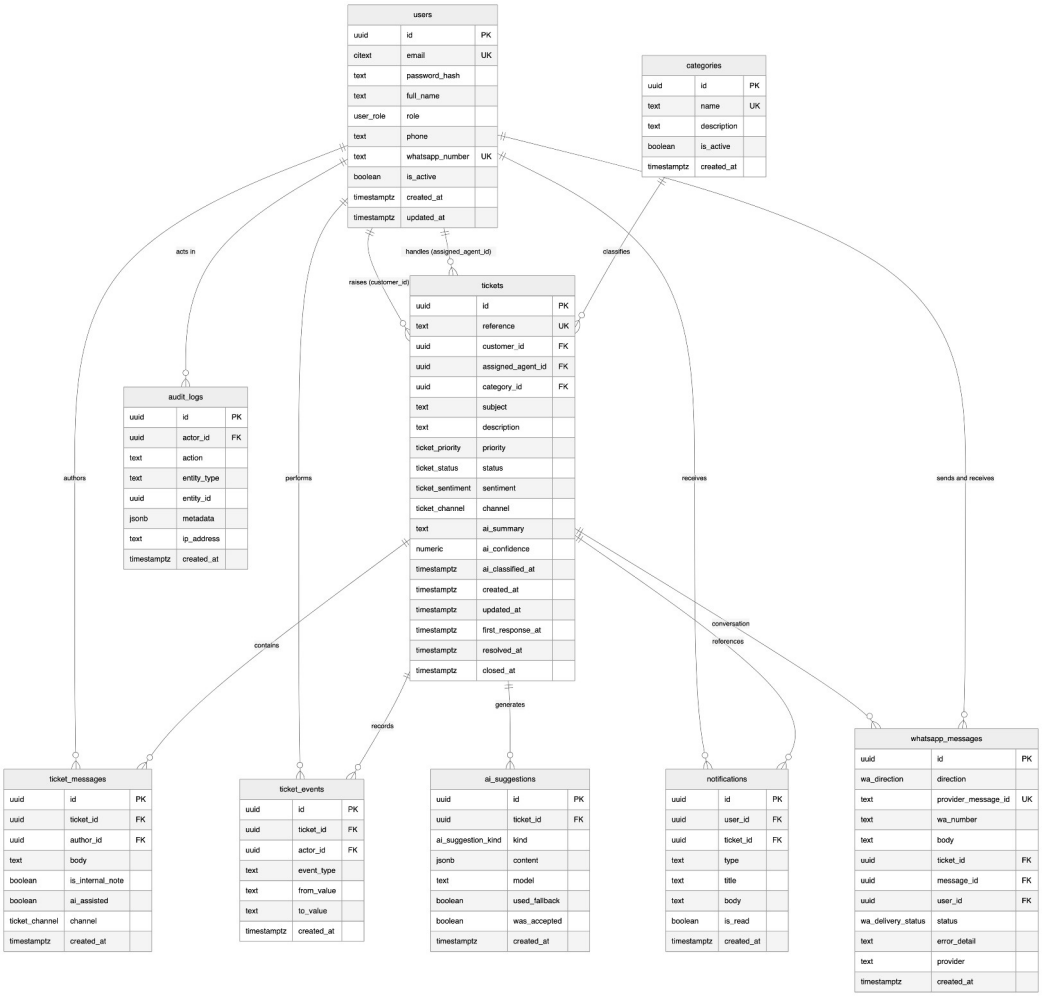



*Figure 11.1 — Entity relationship diagram, derived from the migration files*

## 11.3 Major entities

| Entity | Purpose | Notable design decisions |
|---|---|---|
| tenants | One connected business | Every tenant-owned table references it; a flag controls whether the built-in helpdesk consumes that business's messages |
| users | Every account, all three roles | Case-insensitive unique email; a soft deactivation flag preserves history instead of deleting rows |
| categories | Ticket taxonomy | Unique per business, so two businesses may both have a category named Billing; deletion is refused while tickets reference it |
| tickets | The core entity | A human-readable reference from a sequence; the customer foreign key restricts deletion to protect history while the agent key nulls, returning the ticket to the queue; a CHECK constraint permits a resolution timestamp only in a resolved or closed state |
| ticket_messages | The conversation thread | Internal notes are filtered in SQL rather than in the client, so they cannot leak through the API |
| ticket_events | Per-ticket timeline | Append-only; records who changed what, from which value, and when |
| ai_suggestions | Every AI output | Stores whether the offline fallback produced it and whether an agent accepted it, which keeps reported AI usage honest |
| notifications | Per-user inbox | Scoped by user; another user's notification returns 404 |
| audit_logs | Privileged actions | Actor, action, entity, metadata and address; the record outlives the |

| Entity | Purpose | Notable design decisions |
|---|---|---|
| | | account |
| whatsapp_messages | Channel log | A unique provider message identifier per business makes webhook delivery idempotent; failures are stored with their error so an administrator can see them |
| wa_connections, wa_numbers | Connected businesses | The access token is encrypted; the phone number identifier is unique platform-wide because it is the key by which an inbound message finds its business |
| kb_articles, kb_chunks | Knowledge base | Articles are chunked with overlap and stored with 384-dimension embeddings indexed for cosine similarity |

*Table 11.1 — Major entities*

### 11.4 Indexes and schema management

- B-tree indexes on every column used as a filter or join: status, priority, category, assigned agent, customer, creation time, and the foreign keys of messages and events.
- A GIN index over the text search vector of subject, description and reference, backing keyword search.
- An HNSW index on the chunk embeddings for cosine similarity. An IVFFlat index was tried first and replaced after a test showed it silently returning one row where four matched on a near-empty table; a regression test now pins the behaviour.
- Six migrations applied in filename order by a small runner that records each file, so re-running is safe. The same files run locally, in continuous integration and in the cloud.

## 12. Module Description

| Module | Key capabilities |
|---|---|
| Authentication | Registration, login, token refresh, logout and current-user lookup; a short-lived access token with an httpOnly refresh cookie; deactivation takes effect immediately because the user row is re-read on every request |
| User management | Own profile and password; administrative listing, search and creation; role changes; activation and deactivation, with self-lockout prevented |
| Tickets | The core module: creation, retrieval, update and deletion; the guarded lifecycle; assignment; threaded messages and staff-only internal notes; the timeline; search and filters; background classification |
| Reports and dashboards | Three role-specific dashboards, seven reports and CSV export, all computed from live data |
| Administration | Categories, the audit log, the WhatsApp console and AI health |
| WhatsApp channel | Signature-verified webhook, idempotent delivery handling, the conversation model and keyword commands, the plain-text formatter, provider adapters, and delivery of agent replies and status changes |
| Integration platform | Business onboarding through Embedded Signup, routing of one shared webhook to many businesses, API keys, and signed outbound webhooks with retry and backoff |
| AI assistance | Classification after commit; draft replies, summaries and resolution steps; a provider adapter whose every failure converges on a rule-based fallback, with the outcome recorded |
| Knowledge base | Article management, chunking, embedding, vector retrieval with a lexical fallback, and grounded answers whose citations are verified against the passages actually supplied |

*Table 12.1 — Implemented modules*

## 13. User Interface Design

The interface is built on a hand-written design system rather than a component library: one accent colour, a neutral palette, consistent cards and tables, and explicit loading, empty and error states on every data view. It is responsive from 375 pixels, verified at phone, tablet and desktop widths. The screenshots below were captured from the deployed application at the live URL given in section 21, showing real data from the cloud database.

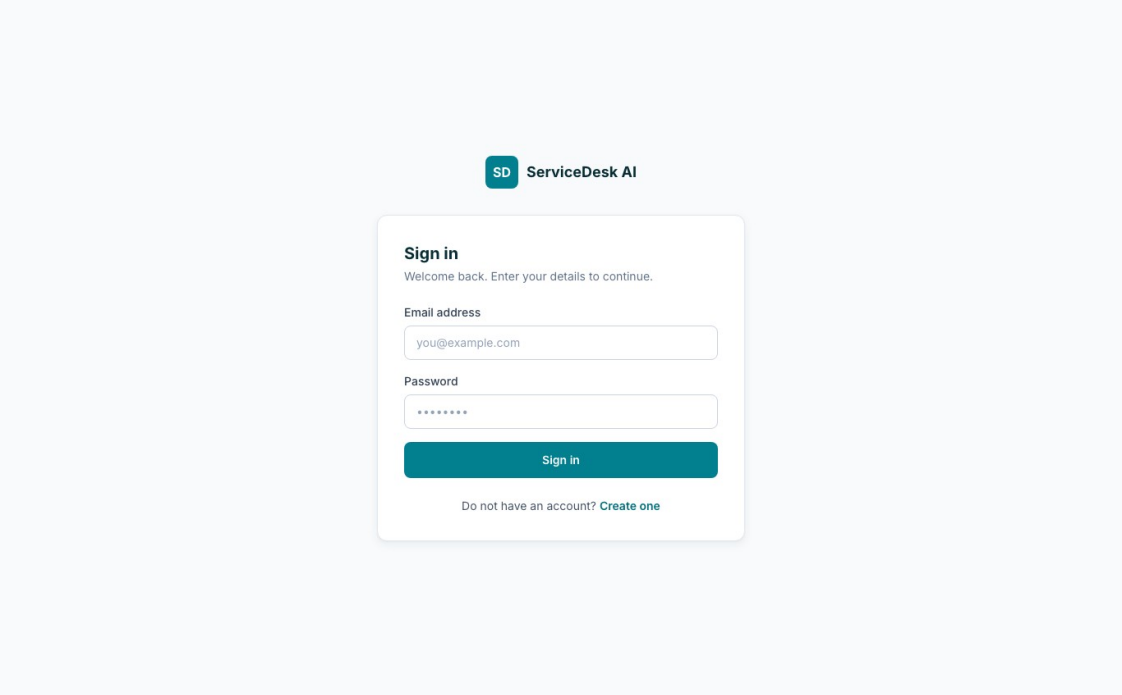



*Figure 13.1 — Sign in*

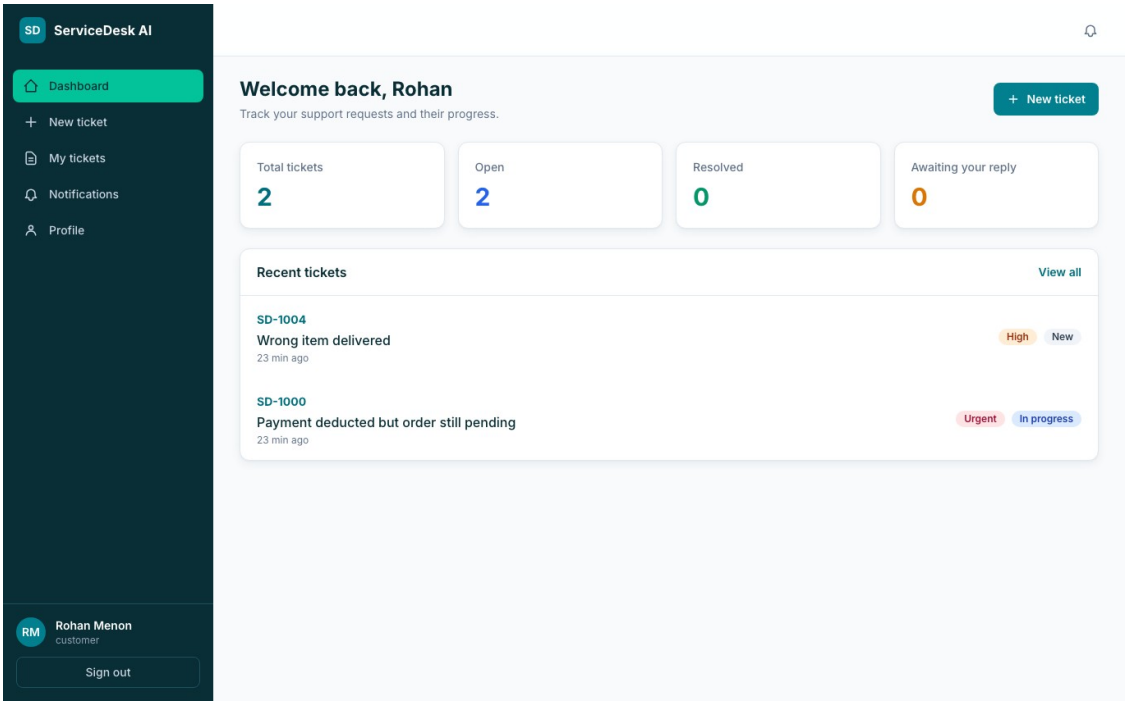



*Figure 13.2 — Customer dashboard, computed from live data*

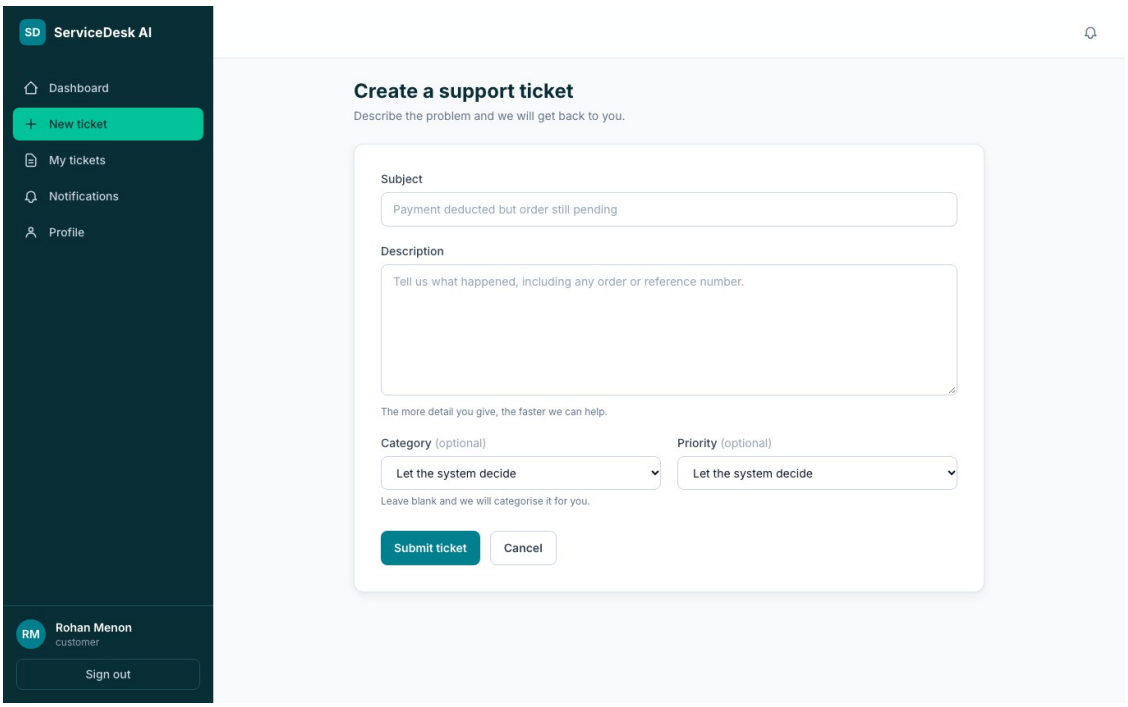



*Figure 13.3 — Raising a ticket*

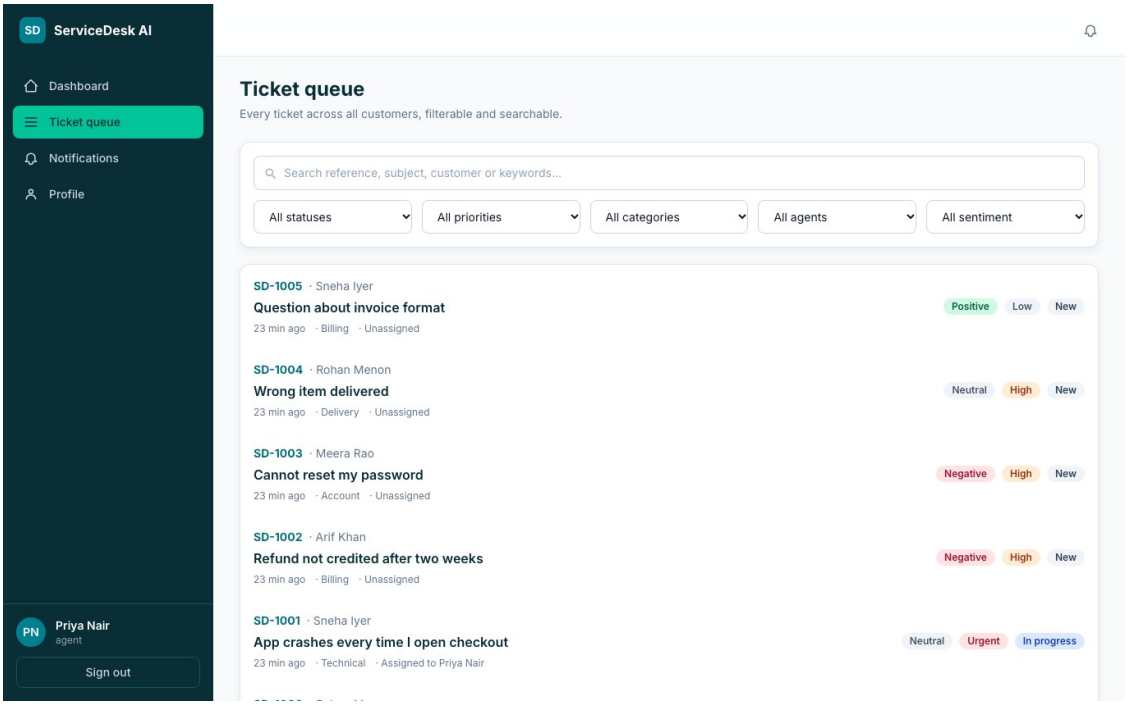



*Figure 13.4 — Agent queue with filters and full-text search*

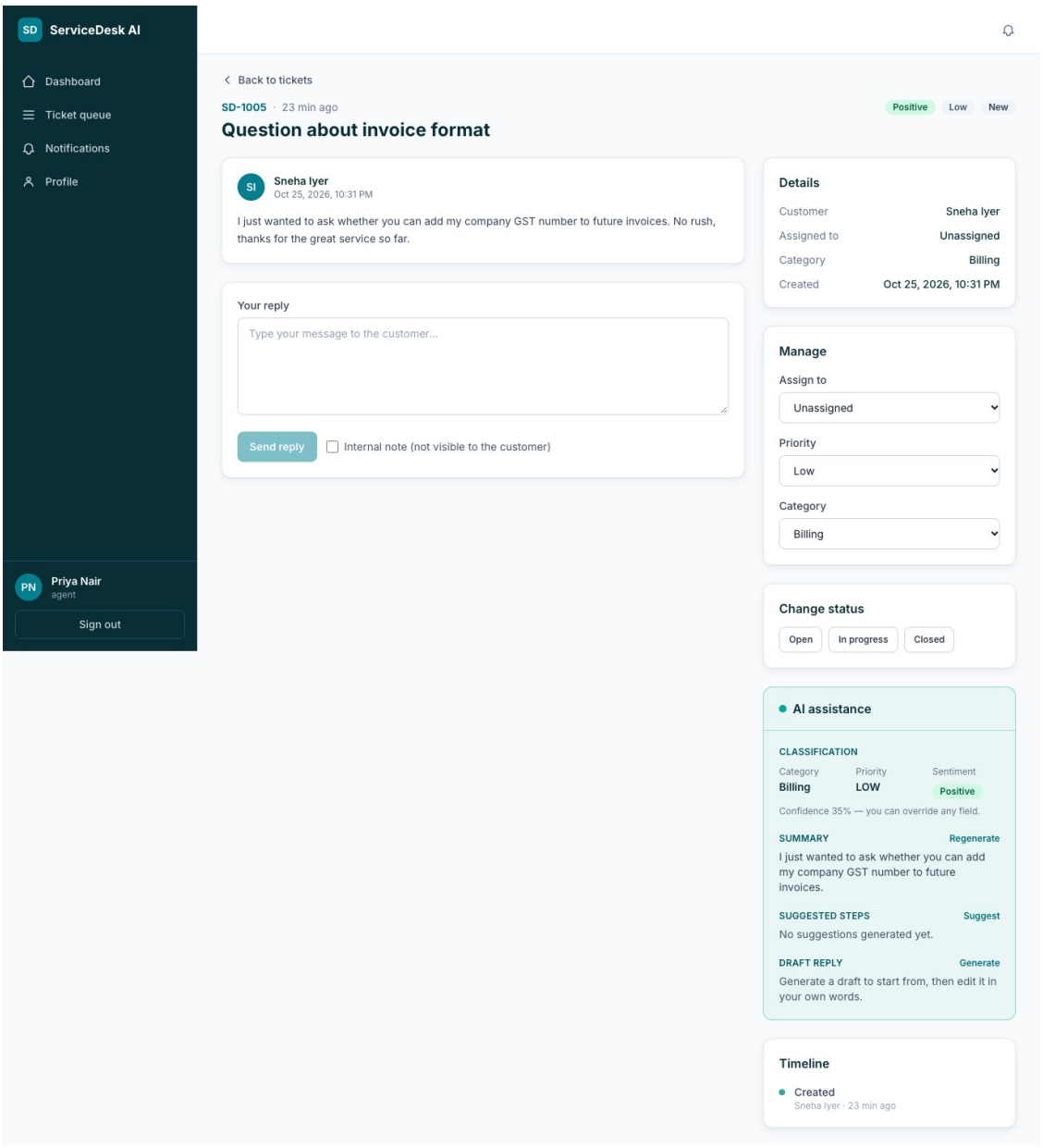



*Figure 13.5 — Ticket detail with the conversation and the AI assistance panel*

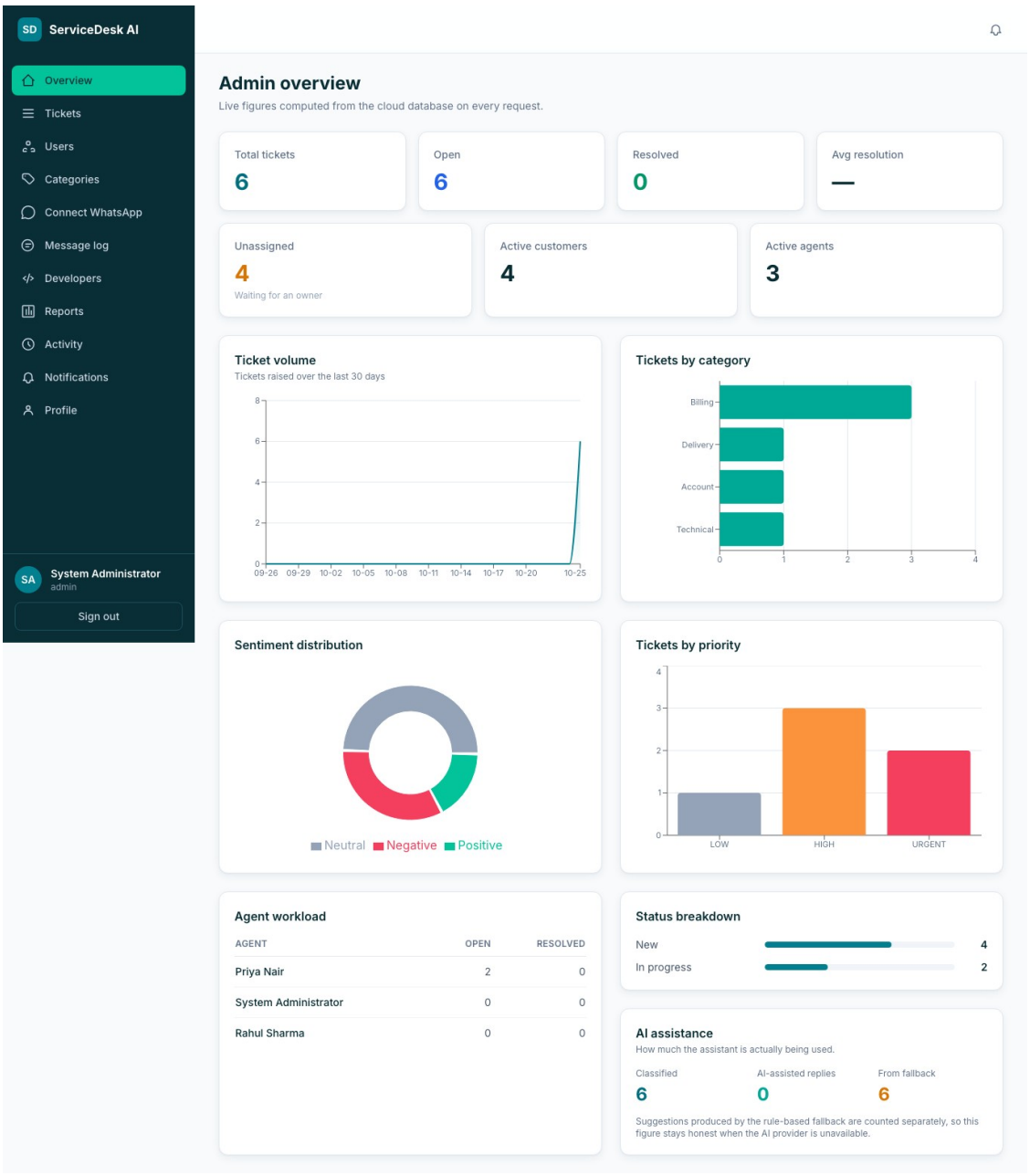



*Figure 13.6 — Administrator dashboard*

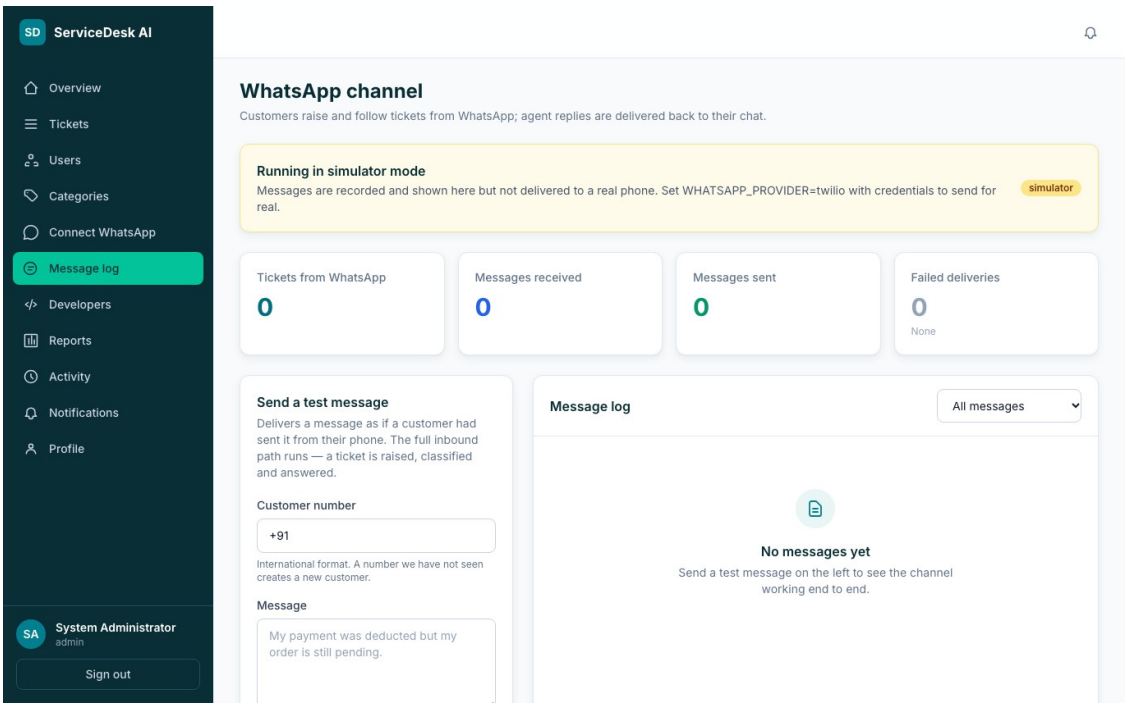



*Figure 13.7 — WhatsApp console: provider status and message log*

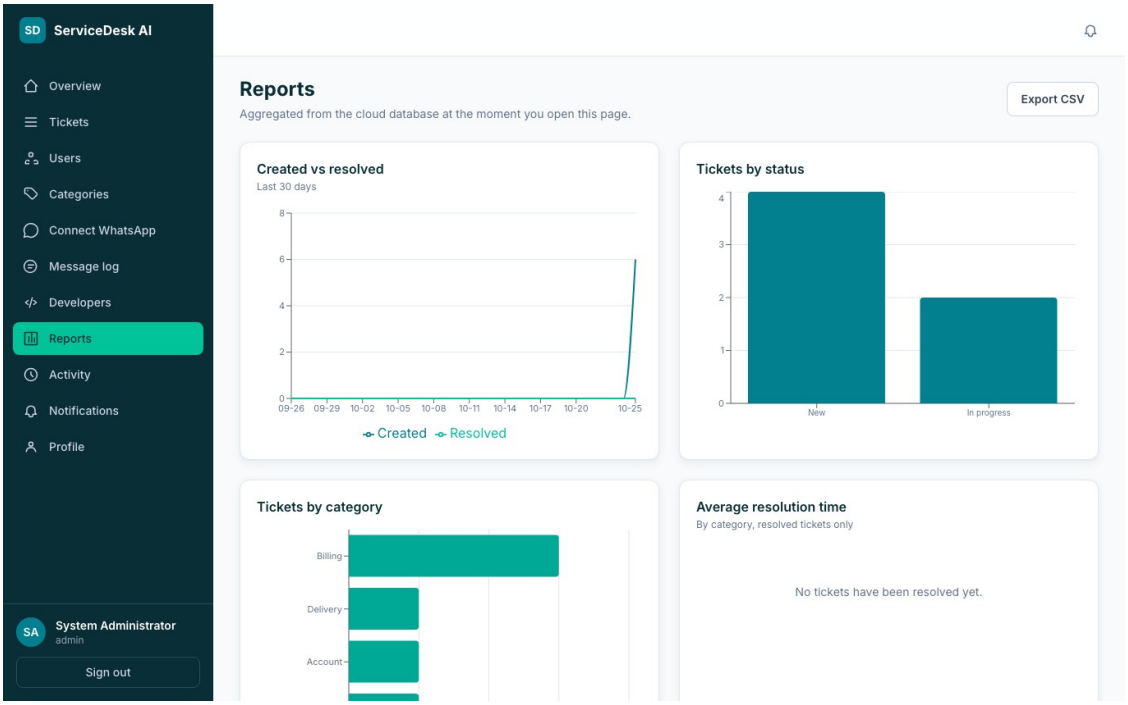



*Figure 13.8 — Reports with CSV export*

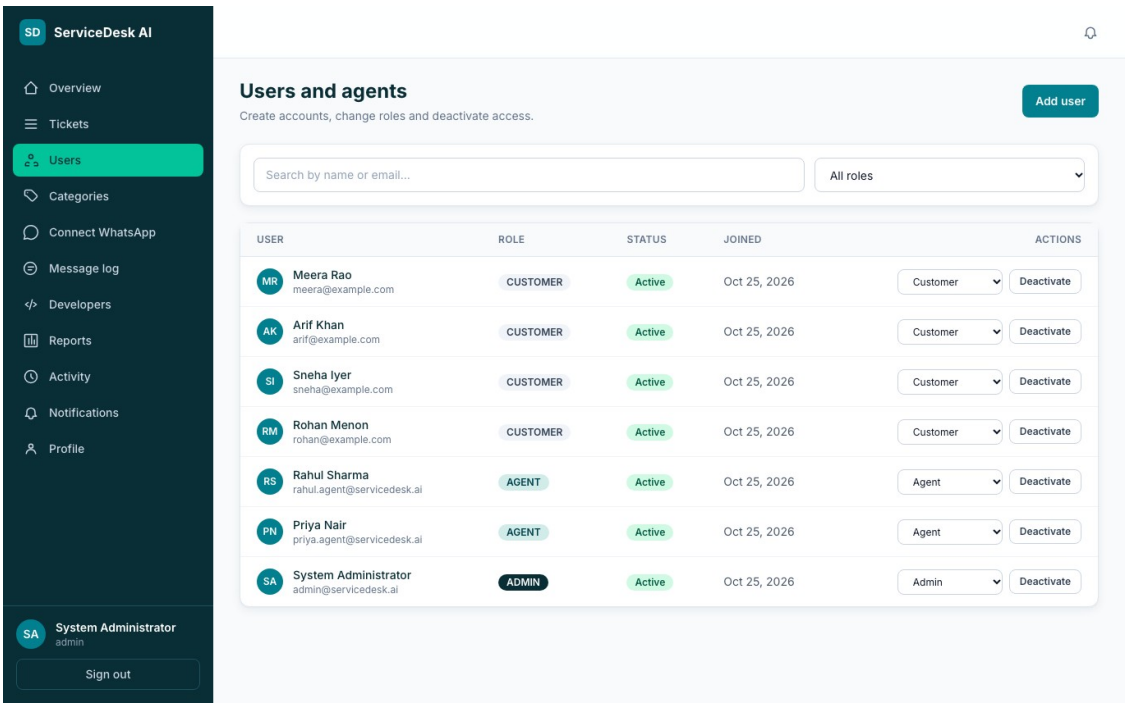



*Figure 13.9 — User management*

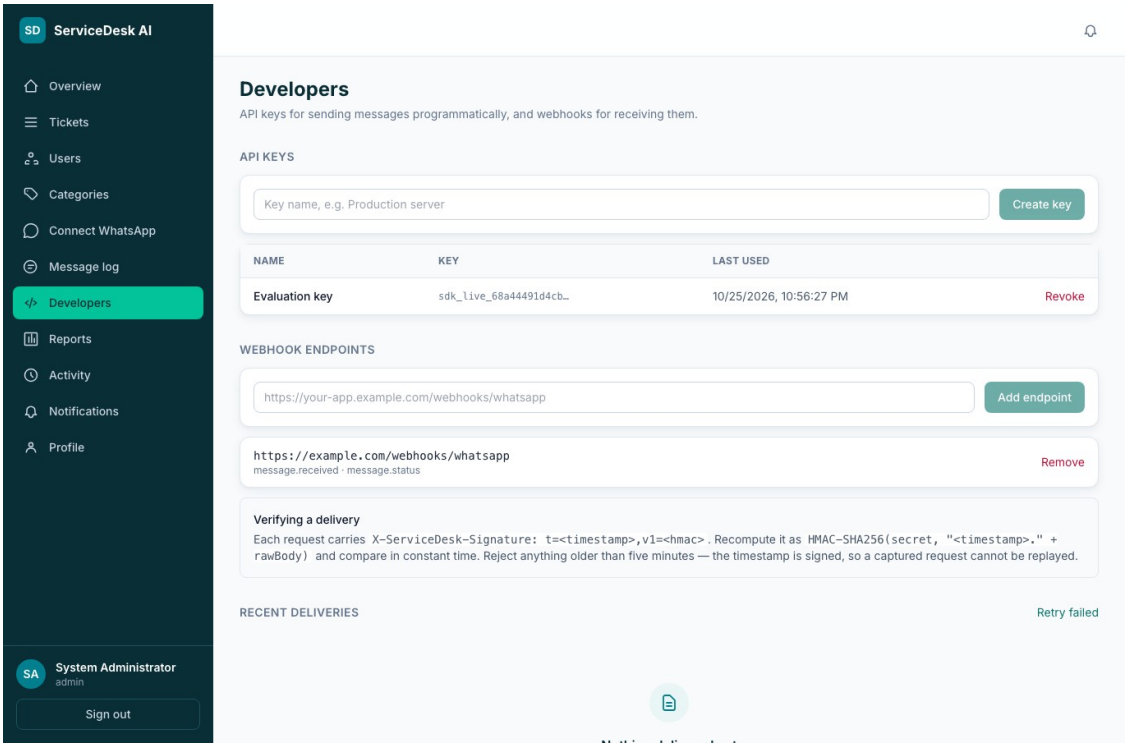



*Figure 13.10 — Developer console: an issued API key, whose "last used" timestamp records a successful call to the public API, and a registered webhook endpoint*

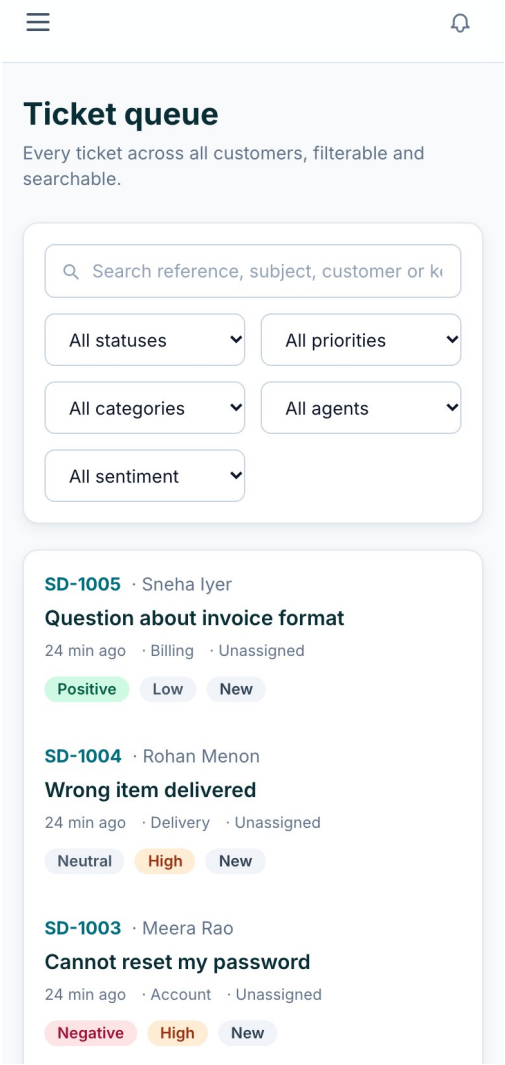



*Figure 13.11 — Responsive layout at 390 pixels*

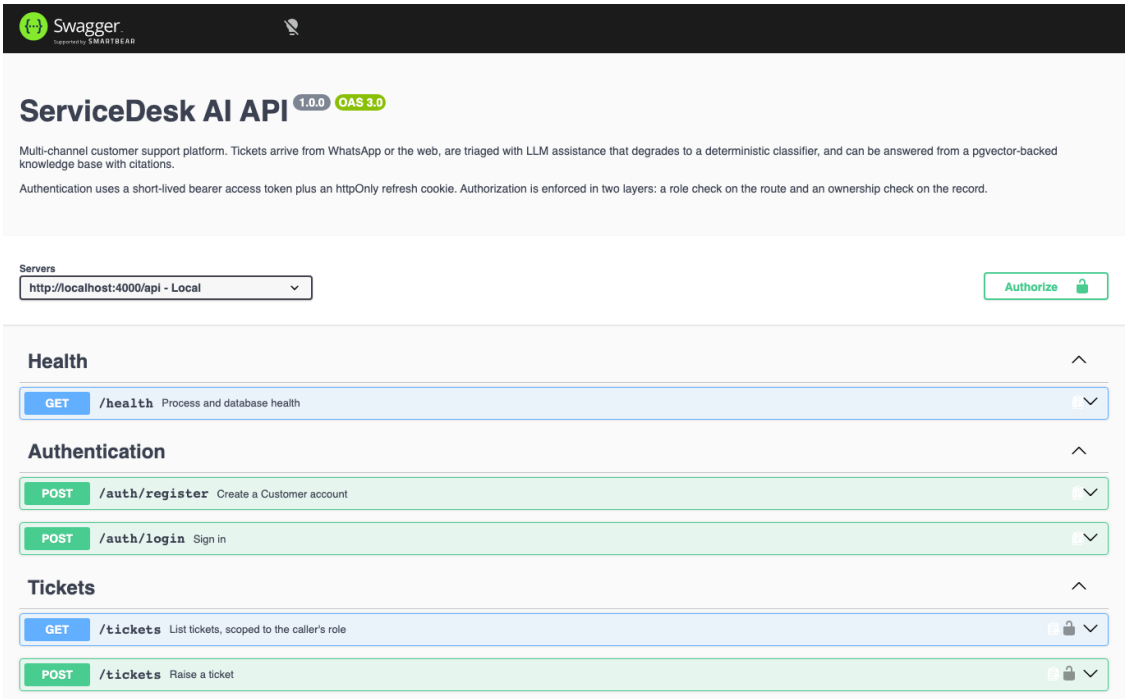



*Figure 13.12 — Interactive API documentation generated from the validation schemas*

# 14. Implementation

## 14.1 Technologies used

| Layer | Technology | Reason |
|---|---|---|
| Frontend | React 18, TypeScript, Vite, Tailwind CSS, TanStack Query, Recharts | A component model with strict typing; a fast static build suited to a CDN; query caching that provides loading and error states consistently |
| Backend | Node.js 20, Express, TypeScript, Zod | Small and explicit, so both team members can read every line; the validation schemas also generate the API documentation |
| Database | PostgreSQL 16 with pgvector | Constraints, full-text search and vector search in one managed engine |
| Authentication | jsonwebtoken, bcryptjs | Stateless sessions that permit replication; a deliberately slow, salted password hash |
| Observability | pino, pino-http | Structured JSON logs with request identifiers and redaction of sensitive fields |
| API documentation | zod-to-openapi, Swagger UI | Generated from the same schemas that validate requests, so documentation cannot drift from behaviour |
| Testing | Vitest, Supertest | Tests at the HTTP boundary against a real database |
| Packaging | Docker multi-stage builds, Docker Compose, nginx | One command brings up the whole stack locally, identically to continuous integration |

*Table 14.1 — Technology stack*

## 14.2 API design

Approximately sixty-four endpoints across eleven groups follow one convention: resource-oriented paths beneath `/api`, JSON bodies validated by Zod, a single response envelope, HTTP status codes used precisely — 201 for creation, 400 for validation failure, 401 unauthenticated, 403 forbidden, 404 for absent or not-yours, 409 conflict, 422 for a business-rule violation and 429 for rate limiting — and pagination metadata on every list. The specification is served as OpenAPI 3 with an interactive Swagger interface.

## 14.3 Security implementation

- Passwords are hashed with bcrypt at cost 12. The hash is never logged, and the login response is identical for an unknown email and a wrong password, so accounts cannot be enumerated.
- A successful login returns a fifteen-minute access token held only in memory by the frontend, never in browser storage, so a cross-site scripting payload cannot read it, together with a seven-day refresh token in an httpOnly cookie scoped to the authentication path.
- Authorisation is enforced twice: route middleware rejects the wrong role before the handler runs, and handlers that touch a specific record check ownership again. A customer requesting another customer's ticket receives 404 rather than 403, so the existence of the record is not disclosed.
- Every inbound webhook is verified against an HMAC signature over the raw request body using a constant-time comparison, before any processing takes place.
- Access tokens held on behalf of connected businesses are encrypted with AES-256-GCM, which authenticates the ciphertext so a tampered row fails to decrypt rather than silently producing wrong bytes.

## 14.4 The AI layer and why it cannot break anything

The specification does not require artificial intelligence; it is an addition, and was therefore designed so that no mandatory requirement depends on it. Classification runs after the ticket has been committed and is not

awaited, so a slow or failing provider cannot delay or fail ticket creation. Every failure — a timeout, a network error, malformed output — converges on a deterministic rule-based classifier, and the result is stored with a flag recording that the fallback produced it, so reported usage stays honest. Finally, the AI never messages a customer: every endpoint returns a suggestion to the agent, and a reply reaches a customer only when an agent sends it. Untrusted ticket text is placed in the prompt as delimited data, and a test confirms that an instruction embedded in a ticket does not change the resulting priority.

### 14.5 Cloud-related components

- A health endpoint that verifies both the process and the database, used by the platform's health check and by external uptime monitoring.
- Graceful shutdown on SIGTERM, so in-flight requests complete when the platform replaces an instance.
- A bounded connection pool, so horizontal replication does not exhaust the managed database's connection limit.
- Migrations executed automatically before the server binds its port, so the schema is always current and a failed migration prevents a broken release from serving traffic.
- Structured JSON logging, which is what managed log aggregation expects.

## 15. Testing

### 15.1 Approach

Tests are written at the HTTP boundary using Supertest against a real PostgreSQL database created from the production migration files, rather than against mocks. This matters: a mock would accept data that a CHECK constraint rejects, so the tests exercise constraints, transactions and SQL exactly as production does. Each file resets the database between cases, and the AI and messaging providers are forced to their offline implementations so results are deterministic and require no credentials. The same suite runs locally, in Docker and in continuous integration on every push.

### 15.2 Results

| | |
|---|---|
| Environment | Node.js 20, PostgreSQL 16 with pgvector, offline AI and messaging providers |
| Command | npm test (Vitest) |
| Test files | 7 |
| Tests | 193 passed, 0 failed |
| Continuous integration | GitHub Actions, green on the default branch |

### 15.3 Coverage by area

| Area | Cases | What is proven |
|---|---|---|
| Authentication | 13 | Registration rules, role forcing on self-registration, password hashing, identical responses for unknown email and wrong password, immediate effect of deactivation, rejection of forged tokens |
| Authorisation | 26 | Every role boundary, exercised by calling the API directly with a valid token for the wrong role; a customer cannot read another customer's ticket; internal notes are hidden; an administrator cannot remove their own role |
| Tickets | 33 | Creation, validation, timeline events, assignment, every lifecycle transition and rejection, first-response and resolution timestamps, search, filters, pagination, notifications and cascading deletion |

| Area | Cases | What is proven |
|---|---|---|
| Lifecycle and AI | 20 | The transition table; classification after commit; fallback on timeout, network error and malformed output; the fallback flag persisted; drafts never sent automatically; resistance to an instruction embedded in a ticket |
| Knowledge base | 31 | Chunking, indexing within a transaction, vector retrieval, lexical fallback, citation verification, the insufficiency flag, and the vector-index regression |
| WhatsApp channel | 38 | Plain-text formatting, number normalisation, provider payload parsing and signature verification, the whole inbound flow including replay suppression and every keyword, outbound delivery of replies and status changes, and masking in the administrative log |
| Integration platform | 32 | Tenant isolation, credential encryption, API-key authentication and revocation, webhook signing, retry with backoff, and account-status handling |

*Table 15.1 — Automated test coverage*

### 15.4 Representative test cases

| Test | Expected | Result |
|---|---|---|
| Register supplying an administrator role | Role forced to Customer | Pass |
| Inspect the stored password | A bcrypt hash, not the password | Pass |
| Compare unknown-email and wrong-password logins | Identical status and message | Pass |
| Use a valid token after the account is deactivated | Refused immediately | Pass |
| Customer requests another customer’s ticket | 404, so existence is not disclosed | Pass |
| Attempt an invalid status transition | Rejected with 422 | Pass |
| Force the AI provider to fail during ticket creation | Ticket still created | Pass |
| Generate a draft reply | No customer-visible message created | Pass |
| Replay an identical webhook delivery | No second ticket raised | Pass |
| Send an unsigned webhook | 403, nothing created | Pass |
| One business lists another’s API keys | Nothing returned | Pass |
| Call the public API with a session cookie | 401 — API keys only | Pass |

*Table 15.2 — Representative test cases and results*

### 15.5 Manual verification

- All three roles exercised end to end in a browser against the deployed application.
- Responsive layout confirmed at 390, 768 and 1366 pixels.
- WhatsApp verified from a real handset: an inbound message raised a ticket and returned an acknowledgement in about one second, the STATUS keyword was answered, and an agent reply sent from the web application arrived on the phone.
- Health, CORS preflight and interactive documentation confirmed from outside the hosting platform.

## 16. Deployment

### 16.1 Platforms used

The frontend is deployed on Vercel and the API and database on Render, both from the project's GitHub repository, with automatic deployment on every push to the default branch. The database and the API are created together from a Blueprint file held in version control rather than configured by hand.

### 16.2 Deployment process

1. The database is created first on Neon, and its connection string is supplied to the Blueprint as a secret. The Blueprint is then applied from the repository, creating the API web service and generating the session signing secrets.
2. The build installs dependencies and compiles TypeScript. Because the production environment omits development dependencies by default, the build command requests them explicitly, as the compiler and the migration runner are among them.
3. On start, the service applies any outstanding migrations and only then binds its port, so the schema is current before new code serves traffic and a failed migration prevents a broken release.
4. Demonstration data is seeded once through an idempotent script.
5. The frontend is imported into Vercel with the frontend directory as its root and a single public environment variable naming the API.
6. The API's allowed-origin setting is updated to the exact frontend address, which is required because the refresh cookie is sent with credentials.
7. An external uptime monitor polls the health endpoint every five minutes, so the free instance does not sleep before an evaluation.

### 16.3 Problems encountered and resolved

| Problem | Resolution |
|---|---|
| The database was created in a different region from the API, so the internal connection string did not resolve | The region was pinned in the Blueprint and the stack re-applied — a one-line change, because the infrastructure is defined in version control |
| The build failed because the production install omitted the TypeScript compiler | The build command now requests development dependencies explicitly |
| Login failed from the browser with a network error | The allowed origin did not match the frontend address exactly; origins are now normalised and a rejection is logged with the configured list rather than surfacing as a server error |
| The messaging provider's trial tier refused free-form replies | A second provider was implemented behind the existing adapter interface; no other code changed |
| Inbound messages were never delivered | The application had not been subscribed to the customer account's webhooks — a step the provider's dashboard does not perform. It is now an explicit, recorded step during onboarding |
| The hosting provider's free managed database is deleted thirty days after creation, taking the API down with it because migrations run at start-up, and replacing it requires a paid plan | The database was moved to a provider whose free plan is permanent and carries the required vector extension. Only the connection string changed: the application reads a connection string and does not depend on which provider issued it |

*Table 16.1 — Deployment problems and their resolutions*

## 17. Results and Discussion

### 17.1 Achieved functionality

All nine mandatory technical requirements, all ten specified functional requirements and all six non-functional requirements are implemented and verified. Beyond the specification, the system provides a two-way WhatsApp channel, a multi-tenant integration platform through which other businesses connect their own numbers, an AI

assistance layer with measured accuracy and an honest offline fallback, a knowledge base with vector retrieval, containerised packaging, continuous integration, generated API documentation and structured logging.

### 17.2 Deployment status

| | |
|---|---|
| Live application | https://servicedesk-ai-two.vercel.app |
| REST API | https://servicedesk-api-cea6.onrender.com/api |
| Interactive API documentation | https://servicedesk-api-cea6.onrender.com/api/docs |
| Database | Neon serverless PostgreSQL with pgvector |
| Repository | https://github.com/ItzGOLLY/servicedesk-ai |
| Hosting cost | Nil — every service on its free tier |

### 17.3 Observed results

| Measure | Result |
|---|---|
| Automated tests | 193 passed, 0 failed |
| Ticket creation | Returns immediately; classification completes asynchronously one to three seconds later |
| WhatsApp round trip | Approximately one second from inbound message to acknowledgement |
| Offline classifier category accuracy | 76.7 per cent, against a 20 per cent majority-class baseline |
| Offline classifier priority within one band | 93.3 per cent |
| Responsive layout | Verified at 390, 768 and 1366 pixels |
| Hosting cost | Nil |

*Table 17.1 — Measured outcomes*

### 17.4 Discussion

Two decisions proved most valuable. Defining the infrastructure in a version-controlled file turned a failed first deployment into a one-line correction and a re-apply. Placing every external dependency behind an adapter with an offline implementation meant that when the first messaging provider's trial tier turned out to refuse free-form replies, switching providers touched one new file and no existing behaviour. The tests, because they run against a real database rather than mocks, caught a vector-index defect that would otherwise have shipped silently.

### 17.5 Limitations

Stated plainly rather than omitted:

- File attachments and cloud object storage are not implemented, so the optional cloud-storage clause of the specification is not satisfied.
- Refresh tokens cannot be individually revoked; deactivating the account is the available mitigation.
- The frontend has no automated tests; it was verified manually across all three roles and three viewport widths.
- The live language-model path is not exercised by the automated suite, because that would require a network call and a credential and would make the suite non-deterministic. Its failure handling is tested by forcing failures.

- Commercial approval to onboard businesses unconnected to the project has not been obtained, as the provider requires verification of a registered company. The integration is complete and demonstrable with permitted test accounts.
- The free database tier expires after thirty days and the free application instance sleeps when idle; both are limitations of the hosting tier rather than of the application.

## 18. Future Enhancements

| Enhancement | Rationale |
|---|---|
| File attachments in cloud object storage | Screenshots make technical issues far faster to diagnose, and this completes the optional cloud-storage requirement |
| Refresh-token rotation with a revocation list | Allows an individual session to be revoked rather than deactivating the whole account |
| Service-level targets with escalation | Response and resolution times are already measured; acting on them turns measurement into management |
| Email as a second channel | Reaches customers who do not use WhatsApp, using the same ticket model |
| A durable queue for outbound delivery | Message delivery and campaign sending currently run in the application process; a queue would suit far higher volume |
| Per-business billing | Usage is already metered; pricing it would make the platform commercially operable |
| Automated frontend and end-to-end tests | The backend is well covered; the interface is verified manually |

*Table 18.1 — Realistic future enhancements*

## 19. References

1. P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” NIST Special Publication 800-145, 2011.
2. PostgreSQL Global Development Group, PostgreSQL 16 Documentation — Full Text Search, Constraints, Indexes. https://www.postgresql.org/docs/16/
3. pgvector — open-source vector similarity search for PostgreSQL. https://github.com/pgvector/pgvector
4. Y. Malkov and D. Yashunin, “Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs,” IEEE TPAMI, 2020.
5. Meta Platforms, WhatsApp Business Platform — Cloud API, Webhooks and Embedded Signup documentation. https://developers.facebook.com/docs/whatsapp/
6. Render, Blueprint specification (render.yaml). https://render.com/docs/blueprint-spec
7. Vercel, Deploying Vite applications. https://vercel.com/docs
8. OpenAPI Initiative, OpenAPI Specification 3.0. https://spec.openapis.org/oas/v3.0.3
9. M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, 2015.
10. N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” USENIX Annual Technical Conference, 1999.
11. OWASP Foundation, Authentication and Session Management Cheat Sheets. https://cheatsheetseries.owasp.org/
12. P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.
13. React, Vite, Tailwind CSS, TanStack Query, Express, Zod, pino and Vitest — official project documentation.

## 20. Appendix

### 20.1 Repository structure

| Path | Contents |
|---|---|
| backend/ | The Express API: modules, services, middleware, migrations, seed data and the test suite |
| frontend/ | The React application, its design system and the deployment configuration |
| docs/ | Architecture, database design, API documentation, testing notes and the deployment runbooks |
| screenshots/ | Interface screenshots captured from the deployed application |
| render.yaml | The Blueprint that provisions the database and the API together |
| docker-compose.yml, docker/ | The complete local stack, matching continuous integration |
| .github/workflows/ci.yml | Continuous integration: typecheck, tests against a real database, and builds |

### 20.2 How to run the project locally

`docker compose up -d` brings up PostgreSQL with the vector extension, applies the migrations, seeds demonstration data and starts both the API and the frontend. Alternatively, with a local PostgreSQL instance available, `npm run migrate && npm run seed && npm run dev` in the backend directory and `npm run dev` in the frontend directory achieve the same result.

## 21. Project Access Details

**Mandatory:** The final application is accessible through the internet. The credentials below are demonstration accounts created specifically for evaluation. They are active, and were verified against the deployed application before submission.

| | |
|---|---|
| Live Application URL | https://servicedesk-ai-two.vercel.app |
| Test User Username / Login | rohan@example.com |
| Test User Password | Customer@12345 |
| Admin Username / Login | admin@servicedesk.ai |
| Admin Password | Admin@12345 |

*The application implements three roles. An additional Support Agent account is provided so that the full workflow, including the agent queue and the AI assistance panel, can be evaluated.*

| | |
|---|---|
| Support Agent Login | priya.agent@servicedesk.ai |
| Support Agent Password | Agent@12345 |

**Suggested evaluation path:** sign in as the administrator to see the dashboards, reports, user management and the WhatsApp console; sign in as the support agent to see the prioritised queue and the AI assistance panel on a ticket; sign in as the test customer, or register a new account, to raise a ticket and watch it classified automatically.

*Credentials provided here are demonstration accounts only. They are not personal passwords and are unrelated to any other account.*

## Final Submission Checklist

| | Item | Status |
|---|---|---|
| ☑ | Final Project Report | This document |
| ☑ | Public Live URL — application accessible through the Internet | https://servicedesk-ai-two.vercel.app |

| | Item | Status |
|---|---|---|
| ☑ | Valid testing login credentials included in the report | Section 21 |
| ☑ | GitHub Repository URL with source code | https://github.com/ItzGOLLY/servicedesk-ai |
| ☑ | PPT / Project Presentation | docs/ServiceDesk_AI_Final_Demo.pptx in the repository |
| ☑ | Working application demonstration | Demonstrated during the semester reviews |
| ☑ | Both team members prepared to explain the implementation | Confirmed |

## Declaration

We confirm that the submitted project is our team's work, that the information and links provided are accurate, and that we can explain the implementation during evaluation. We understand the project guidelines regarding academic integrity and responsible use of AI tools.

| | |
|---|---|
| Name | Aarush Jagannathan — 23BLC1211 |
| Name | Gomatheswar M — 23BLC1195 |
| Faculty | Dr. M Dinakaran |
| Date | |
| Signatures | |